# Computational Complexity

# Guideline 1

The running time of a comments, declarative and function statements count to zero

Declarative Statements

```
template <class Type>
class List ;
int age;
```

comments

```
//a variable to store age
```

Function statements

```
void Print();
```

Total time = zero

# Guideline 2

Expressions, memory management & assignment statements for primitive data types have constant running time i.e. one.

Expressions and assignment statements

```
int a = 4+5;
char  b ='a';
```

Memory Management statements

```
int * a = new int;
delete a;
```

Total time = constant

# Guideline 3

Memory management & assignment statements for objects have constant running time i.e. size of object.

Memory Management statement of object

```
Student a,b;
Student * a = new Student;
delete a;
```

Assignment statement of object

```
a = b;
```

Total time = constant

# Guideline 4

Function invocations count as 1 step unless the invocation involves pass by value parameter whose size depends on the instance characteristics i.e. sizes of the value parameters

Total time = constant

# Guideline 5: Loops

The running time of a loop is, at most, the running time of the statements inside the loop (including tests) multiplied by the number of iterations.

executed *n* times

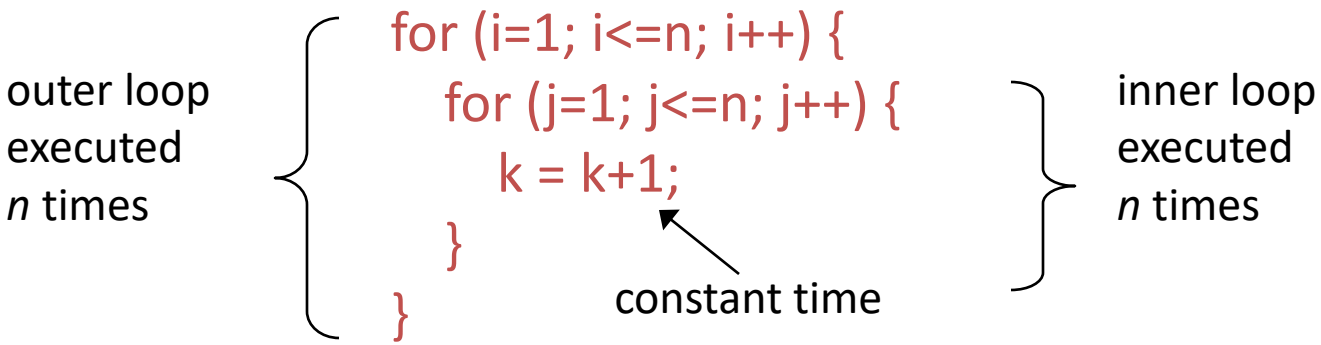
```
for (i=1; i<=n; i++)
{
    m = m + 2;
}
```

constant time

Total time = (constant) c * n = cn

# Guideline 6: Nested loops

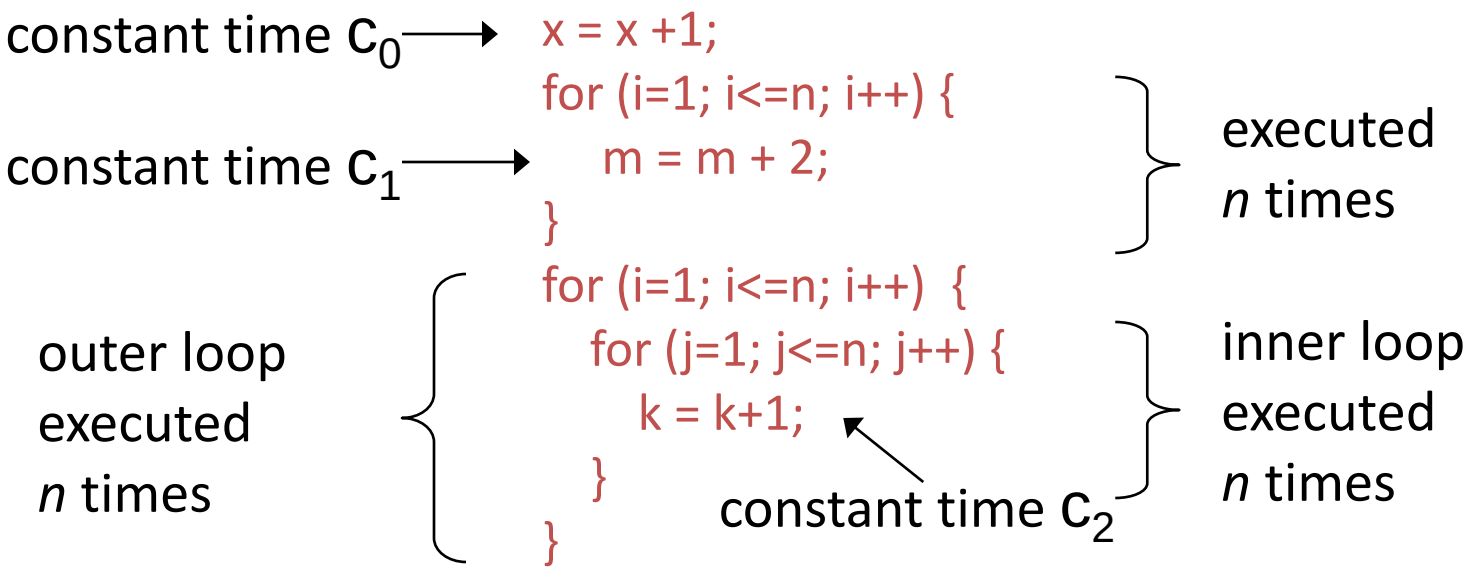
Analyse inside out. Total running time is the product of the sizes of all the loops.




Total time = c * n * n * = $cn^2$

# Guideline 7: Consecutive statements

Add the time complexities of each statement.




Total time = $c_0 + c_1 n + c_2 n^2$

# Guideline 8: If-then-else statements

```
if( exp)
        statements1;
else
        statements2;
```

Cost is the number of steps corresponding to exp, statements1 and statements2.

If exp:
Constant $c_0$

```
if (depth( ) != otherStack.depth( ) ) {
  return false;
}
else {
  for (int n = 0; n < depth( ); n++) {
  if (!list[n].equals(otherStack.list[n]))
    return false;
  }
}
```

Statements1:
Constant $c_1$

**else** part:
(constant $c_2$ +
Constant $c_3$) * n

another if :
constant +
constant
(no else part)

Total time = $c_0 + c_1 + (c_2 + c_3) * n$

# Problems with T(n)

- T(n) is difficult to calculate
- T(n) is also not very meaningful as step size is not exactly defined
- T(n) is usually very complicated so we need an approximation of T(n)….close to T(n).
- This measure of efficiency or approximation of T(n) is called ASYMPTOTIC COMPLEXITY or ASYMPTOTIC ALGORITHM ANALYSIS

*Asymptotic complexity studies the efficiency of an algorithm as the input size becomes large*

# Example

- If T(n) = 7n+100
- What is T(n) for different values of n???

| n | T(n) | Comment |
|---|---|---|
| 1 | 107 | Contributing factor is 100 |
| 5 | 135 | Contributing factor is 7n and 100 |
| 10 | 170 | Contributing factor is 7n and 100 |
| 100 | 800 | Contribution of 100 is small |
| 1000 | 7100 | Contributing factor is 7n |
| 10000 | 70100 | Contributing factor is 7n |
| $10^6$ | 7000100 | What is the contributing factor???? |

*When approximating T(n) we can IGNORE the 100 term for very large value of n and say that T(n) can be approximated by 7(n)*

# Example 2

- $T(n) = n^2 + 100n + \log_{10}n + 1000$

| n | T(n) | $n^2$ | | 100n | | $\log_{10}n$ | | 1000 | |
|---|---|---|---|---|---|---|---|---|---|
| | | Val | % | Val | % | Val | % | Val | % |
| 1 | 1101 | 1 | 0.1% | 100 | 9.1% | 0 | 0% | 1000 | 90.8% |
| 10 | 2101 | 100 | 5.8% | 1000 | 47.6% | 1 | 0.05% | 1000 | 47.6% |
| 100 | 21002 | 10000 | 47.6% | 10000 | 47.6% | 2 | 0.99% | 1000 | 4.76% |
| $10^5$ | 10,010,001,005 | $10^{10}$ | 99.9% | $10^7$ | .099% | 5 | 0.0% | 1000 | 0.00% |

*When approximating T(n) we can IGNORE the last 3 terms and say say that T(n) can be approximated by $n^2$*

# Big-Oh

- **Definition:**

> *f(n) is O(g(n)) if there exist positive numbers c & N such that f(n)<=cg(n) for all n>=N*

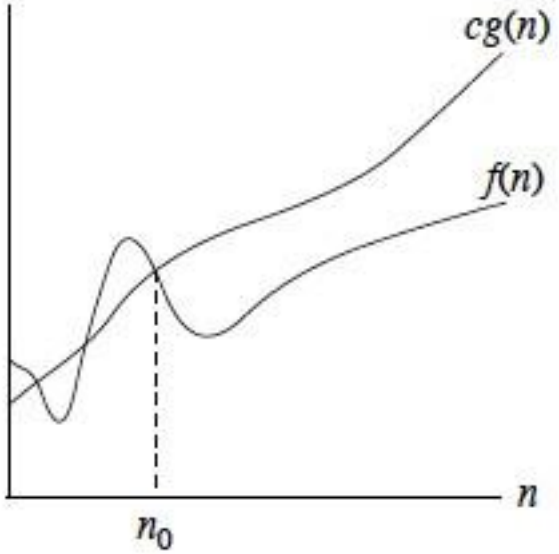
g(n) is called the upper bound on f(n) OR

f(n) grows at the most as large as g(n)

**Example:**

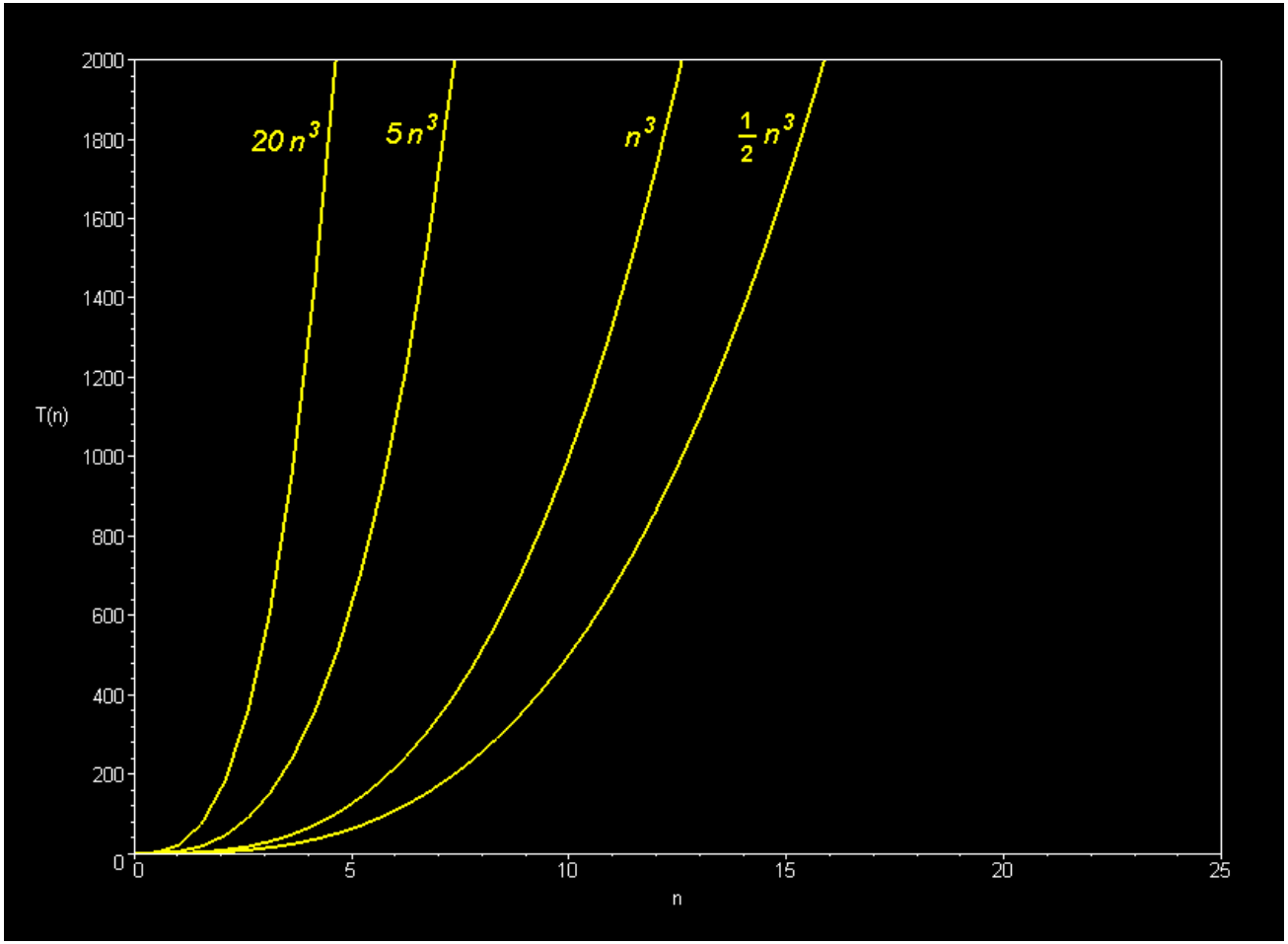
$T(n) = n^2 + 3n + 4$

$n^2 + 3n + 4 <= 2\ n^2$ for all n>10




so we can say that T(n) is $O(n^2)$ OR
T(n) is in the order of $n^2$.

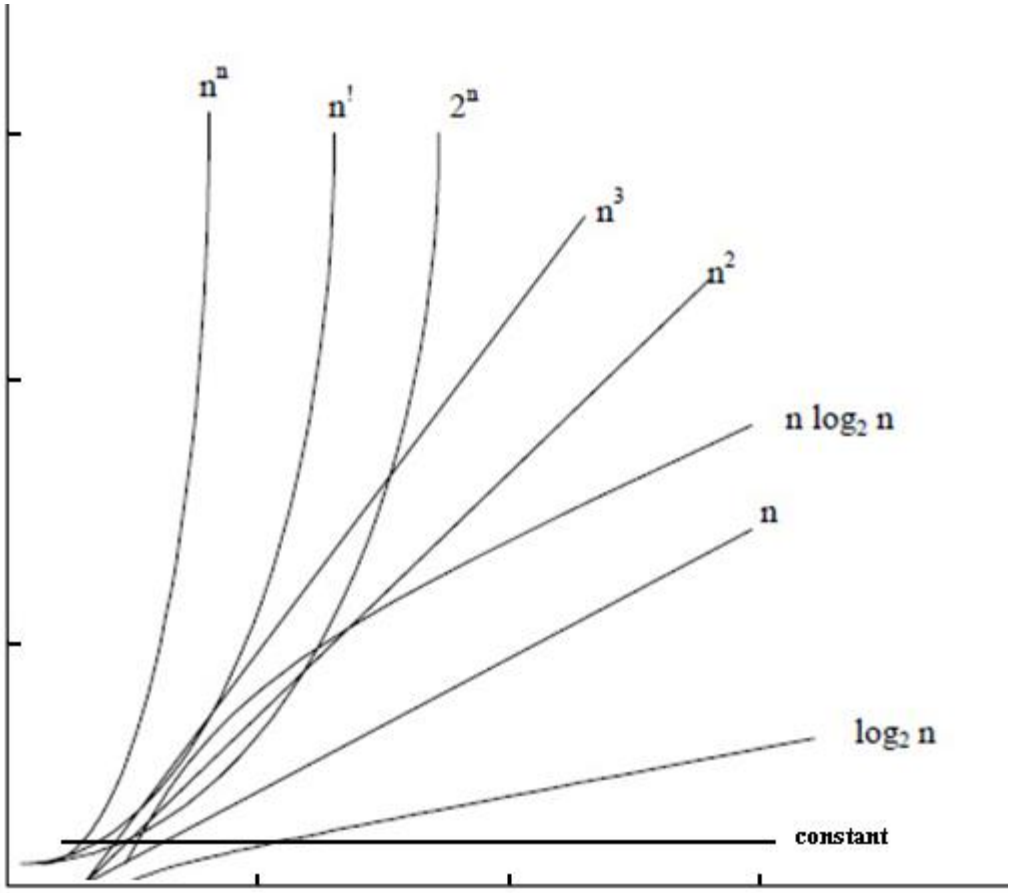
T(n) is bounded above by a + real multiple of $n^2$


2000
1800
1600
1400
1200
1000
800
600
400
200
0
T(n)
0
5
10
15
20
25
n
20 n^3
5 n^3
n^3
1/2 n^3

# Properties of Big-Oh

- If f(n) is O(g(n)) and g(n) is O(h(n)) then f(n) is O(h(n))
- If f(n) is O(h(n)) and g(n) is O(h(n)) then f(n) + g(n) is O(h(n))
- $an^k$ is $O(n^k)$ where a is a constant
- $n^k$ is $O(n^{k+j})$ for any positive j
- If f(n)=cg(n) then f(n) is O(g(n))

# Growth rates
# Which growth rate is better???




Graph from Adam Drozdek's book

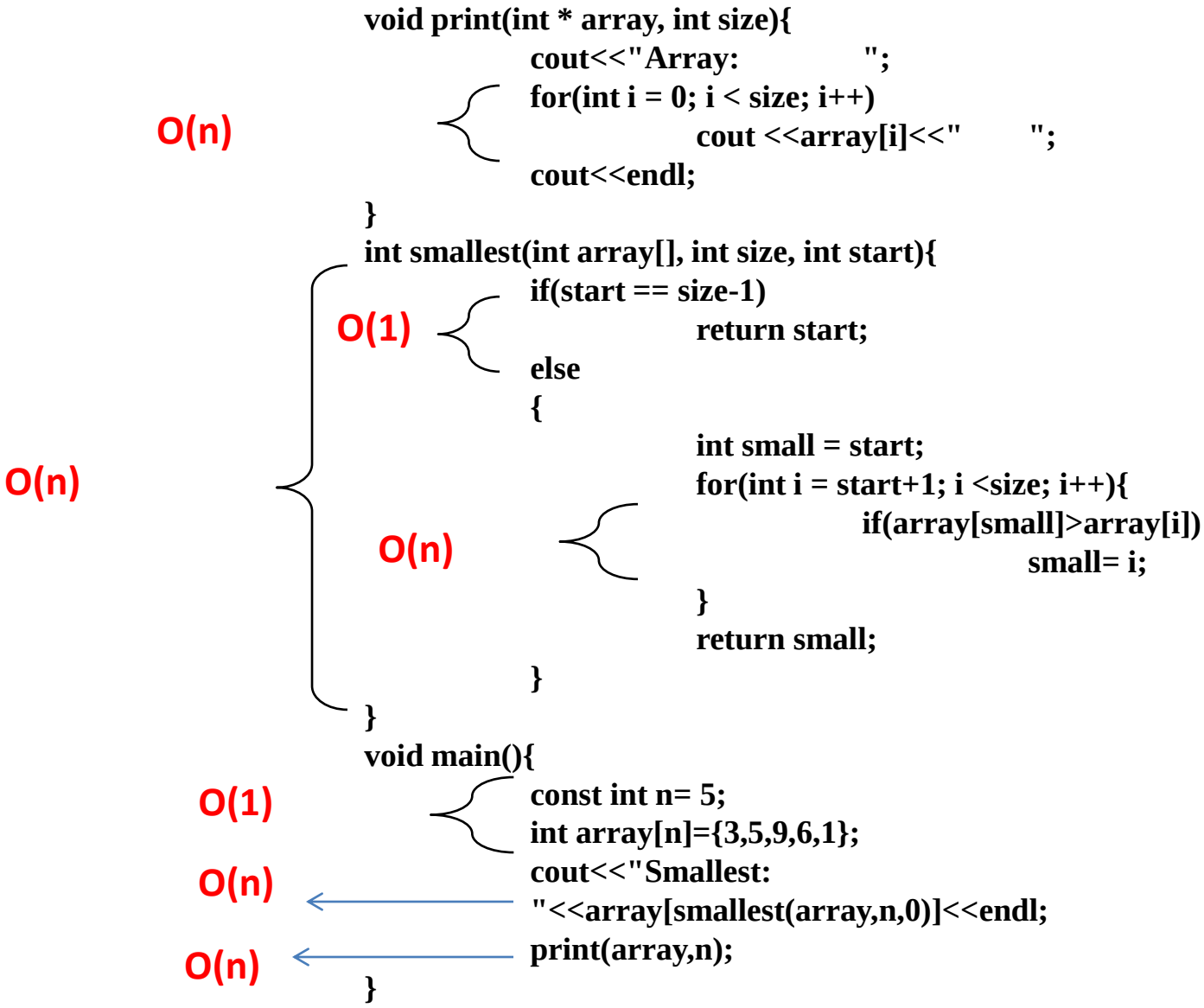
# Comparison of Growth Rates

| N | $\log_2 N$ | $N \log_2 N$ | $N^2$ | $N^3$ | $2^N$ |
|---|---|---|---|---|---|
| 1 | 0 | 0 | 1 | 1 | 2 |
| 2 | 1 | 2 | 4 | 8 | 4 |
| 8 | 3 | 24 | 64 | 512 | 256 |
| 64 | 6 | 384 | 4096 | 262,144 | 18446744073709551616 |
| 128 | 7 | 896 | 16,384 | 2,097,152 | Approx 6 billion years, 600,000 times more than age of univ. |

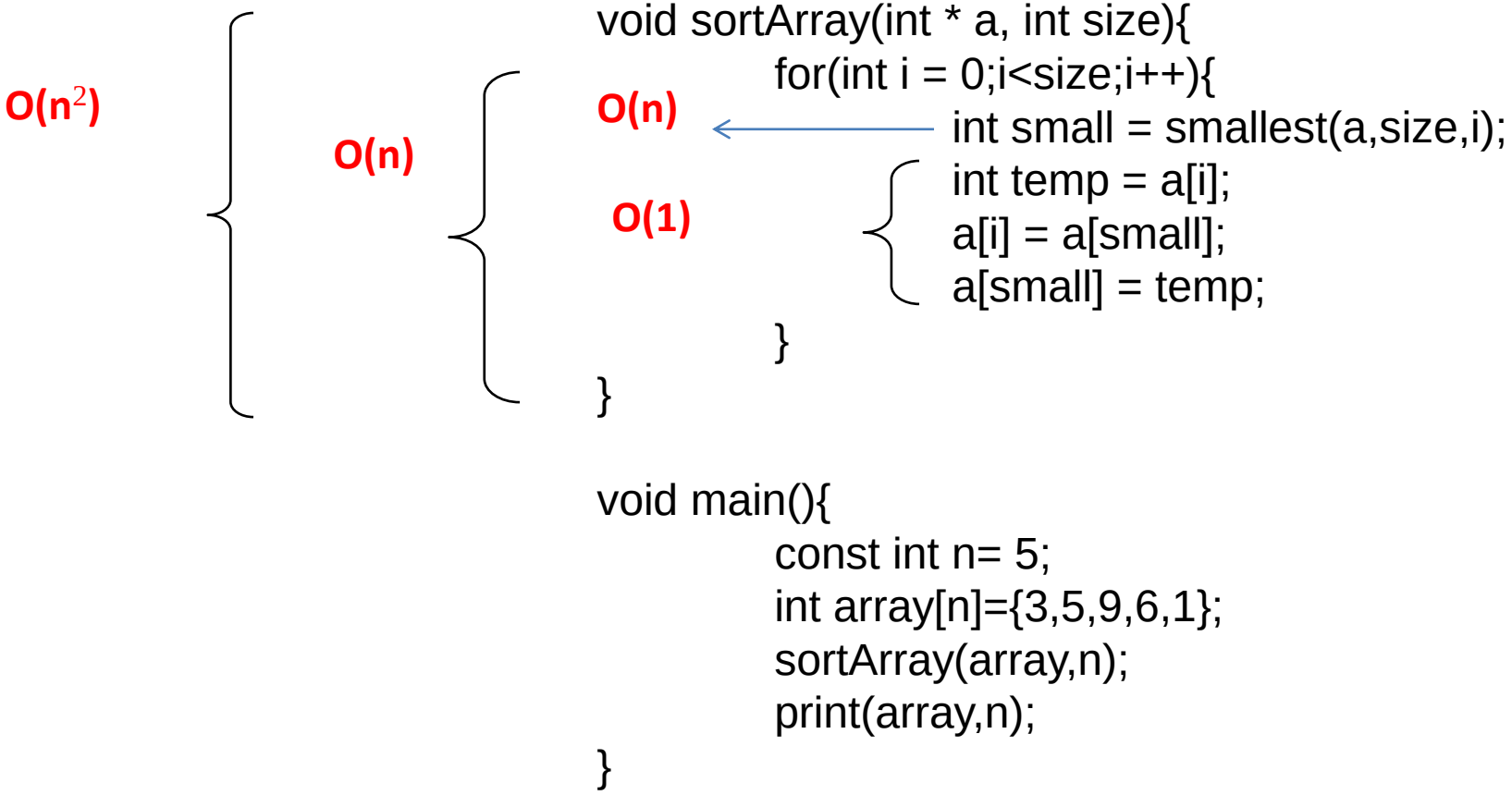
(If one operation takes $10^{-11}$ seconds)

# Big O??

**Running Time: O(1)+O(n)+O(n)= O(n)**

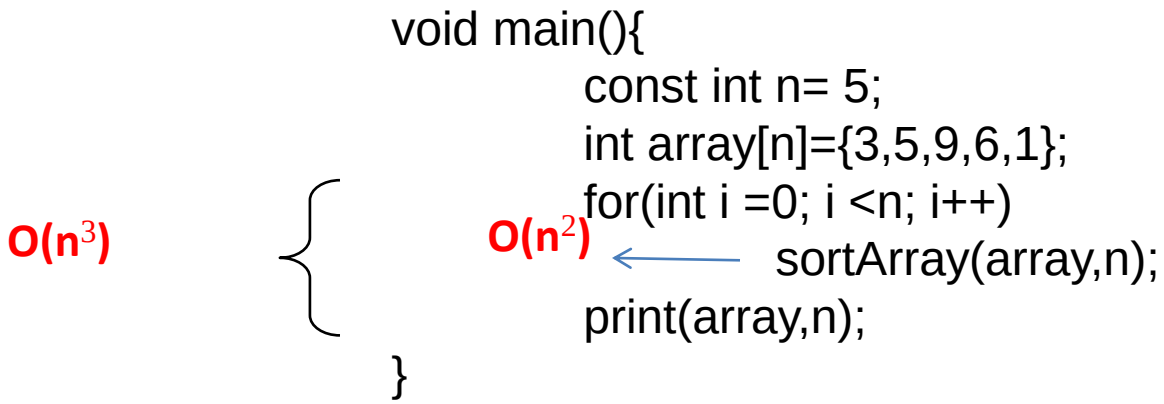


# Big-O?




**Running Time:** $O(1)+O(n^2)+O(n)= O(n^2)$

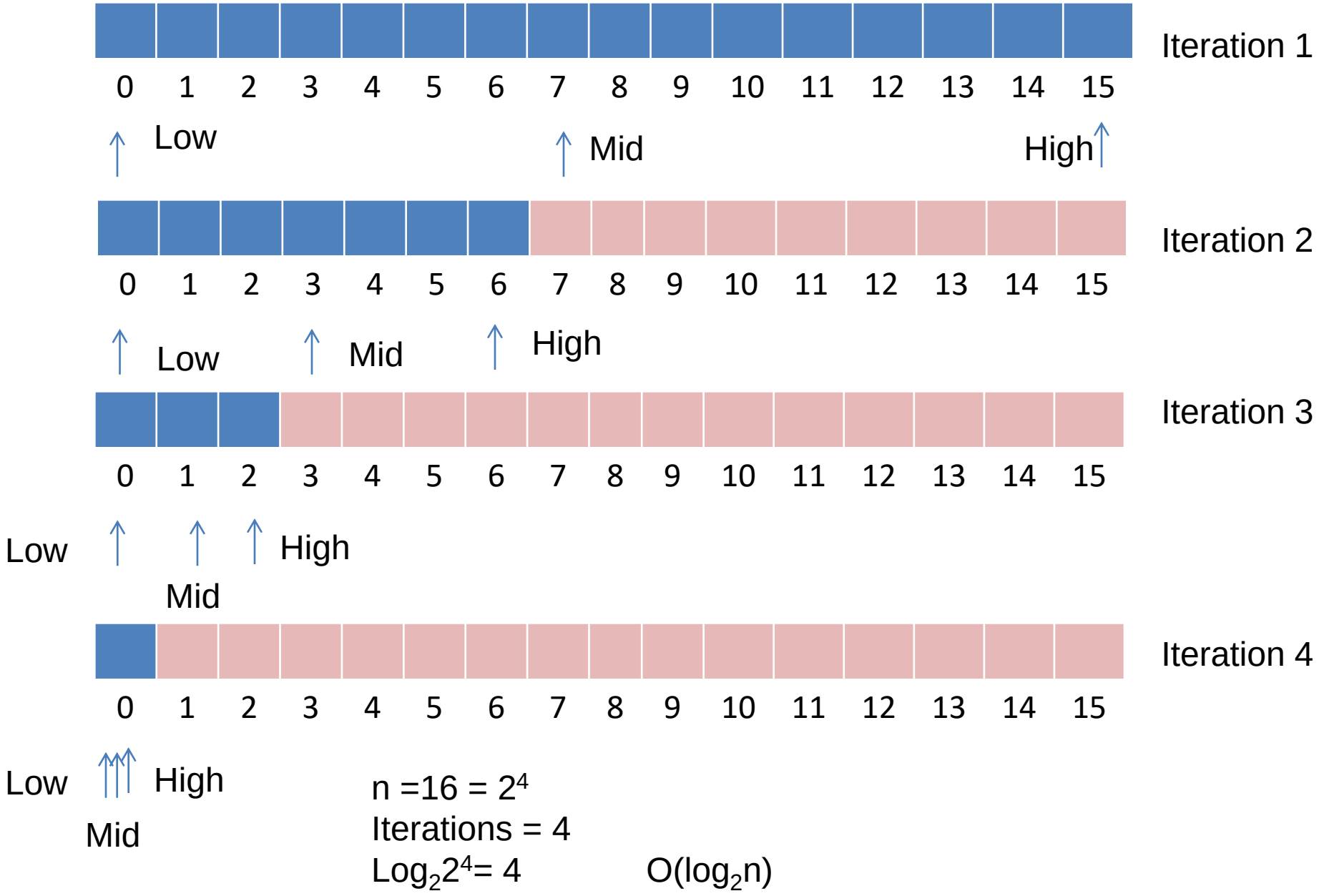
# Big-O?




**Running Time:** $O(1)+ O(n^3) = O(n^3)$

# Big-O?

```
bool BinarySearch(int * a,int size, int search){
        int high = size-1;
        int low = 0;
        int mid ;
        while (low<=high){
                mid = (high+low)/2;
                if(a[mid]==search)
                        return true;
                else if(search < a[mid]){
                        high = mid-1;
                }
                else
                        low = mid+1;
        }
        return false;
}
```


Iteration 1
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
Low
Mid
High
Iteration 2
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
Low
Mid
High
Iteration 3
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
Low
Mid
High
Iteration 4
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
Low
Mid
High
n =16 = 2^4
Iterations = 4
Log2 2^4= 4
O(log2n)